

System Identification of a Nonlinear Two-Mass Spring-Damper System: Least Squares vs. Recurrent Neural Network

In this lab rotation I investigated system identification of a nonlinear two-mass, two-spring, two-damper mechanical system, comparing classical least-squares (LS) against Recurrent Neural Networks (RNNs), motivated by the framework in [1]. The system models two carts connected in series to a fixed wall, each coupling via a nonlinear *hardening* spring with piecewise-linear profile $\gamma(d)$ and a viscous damper (see Appendix A for all governing equations). An external piecewise-constant force $u(t)$ drives Cart 1. The state vector is $\mathbf{x} = [x_1, \dot{x}_1, x_2, \dot{x}_2]^\top$ with $m = [\frac{1}{4}, \frac{1}{3}]$ kg, $c = [\frac{1}{4}, \frac{1}{3}]$ Ns/m, $k = [1, \frac{5}{6}]$ N/m (matching the reference paper). All code is at [2].

Phase 1 — Data Generation & LS Baselines. Two spring variants were simulated (Fig. 1): the *original* system with soft-zone threshold $\delta=1.0$, and a *shrunk* variant with $\delta=0.25$ (supervisor’s suggestion), where the spring enters its stiff linear regime at much smaller displacements. For each variant a training trajectory (seed 42, Fig. 2a–b) and a held-out test trajectory (seed 99) were generated via RK4 ($\Delta t=0.01$ s, $T=200$ s, 20 000 steps). Both raw states $\mathbf{x}$ and relative coordinates $\mathbf{z} = [x_1 - x_2, \dot{x}_1 - \dot{x}_2, x_2, \dot{x}_2]^\top$ were used. Model quality is measured by $\text{NSE} = \|\hat{y} - y\| / \|y\|$ (lower is better; $\text{NSE} > 1$ means divergence). The ARMA predictor $\mathbf{x}_{k+1} = D\mathbf{x}_k + E_1 u_{k+1} + E_2 u_k$ was fit by pseudoinverse on all four combinations (Table 1, Appendix A). On the *original* springs the LS model diverges in closed-loop rollout regardless of coordinate system (test NSE 0.20 raw, 0.31 transformed; Figs. 3–4), as a linear map cannot capture the soft zone’s saturating nonlinearity. Applying the same procedure to the *shrunk* springs yields substantially lower NSE (test 0.08 raw, 0.15 transformed; Figs. 5–6): with $\delta=0.25$ the system spends most of its time in the stiff linear regime, making the global linear approximation much more accurate.

RNN Exploration — Open-Loop to Closed-Loop. A vanilla open-loop Elman RNN ($h_k = \tanh(h_k W_{hh} + u_k W_{uh} + b_h)$, hidden size 32, 3 000 epochs) tracked the ground truth given true inputs at each step (Fig. 7) but diverged immediately in autonomous rollout — errors compound through the feedback path. I redesigned training as a *closed-loop autoregressive* scheme: the model receives $[\mathbf{z}_k; u_k] \in \mathbb{R}^5$, predicts $\hat{\mathbf{z}}_{k+1}$, and that prediction feeds back as the next input. Working in transformed $\mathbf{z}$ -coordinates (hidden 64, batch 16, Adam lr= 10^{-4} , gradient clipping), I compared four activations: **ReLU** (Fig. 8–9), **ELU** (Fig. 10), **tanh** (Fig. 11), and a physics-inspired **deadzone** variant (*NLT*): before the hidden update, z_1 is replaced by $\text{deadzone}(z_1, 1.0)$, zeroing the soft spring region to expose the underlying stiff dynamics more directly to the network (Fig. 12).

Phase 2 — LR Scheduler & Physics-Informed Residual. Experiment 2a (Fig. 13–14) retained the best closed-loop ReLU configuration (h=64, z-coords) but added a ReduceLROnPlateau scheduler (initial lr=0.01, factor 0.5, patience 50 epochs). Training loss reached a minimum of 1.47×10^{-4} over 1 000 epochs, with the LR annealing from 0.01 down to 3.9×10^{-5} across seven reductions. Despite this, autoregressive NSE exceeded 1.7 on both train and test (divergence) — demonstrating a sharp *train/inference gap*: aggressive one-step teacher-forcing with a decaying LR over-optimises short-horizon loss but produces a closed-loop system that is not autonomously stable. **Experiment 2b** (Fig. 15–16) addressed this with a physics-informed residual architecture: the 9-dim input $[\mathbf{z}_k; u_k; \hat{\mathbf{z}}_{k+1}^{\text{LS}}]$ augments the state with the LS model’s forward prediction, so the RNN need only learn the nonlinear residual $\Delta \hat{\mathbf{z}}_{k+1} = \hat{\mathbf{z}}_{k+1} - \hat{\mathbf{z}}_{k+1}^{\text{LS}}$. With the same LR scheduler the model converged stably, reaching test NSE $z_1=0.40$, $z_3=0.20$ — the best RNN result, with z_3 matching the LS baseline on the absolute coordinate.

Summary. The experiments establish a clear capability hierarchy: LS is the most efficient baseline and optimal for near-linear regimes; closed-loop training is *necessary* but not *sufficient* for autonomous stability; and augmenting the RNN input with the LS prediction (Exp. 2b) is the most effective approach tested. The central lesson is that low one-step training loss does not imply closed-loop stability — the training objective must reflect the deployment condition. Future work will pursue robustness certification via the Incremental Quadratic Constraint framework of [1], and extension to the full four-mass system of the reference.

References

- [1] M. Revay, R. Wang, and I. R. Manchester, “A convex parameterization of robust recurrent neural networks,” *IEEE Control Systems Letters*, vol. 5, no. 4, pp. 1363–1368, Oct. 2021.
- [2] O. Gundra, “RNN Honours Rotation,” GitHub, 2025. [Online]. Available: <https://github.com/BraveKing15/RNN-Honours-Rotation>

Appendix A: Governing Equations and Results Summary

Nonlinear Spring Profile ($\delta = \text{threshold}$):

$$\gamma(d; \delta) = \begin{cases} d + 0.75\delta & d \leq -\delta \\ 0.25d & |d| < \delta \\ d - 0.75\delta & d \geq \delta \end{cases} \quad (\text{original: } \delta=1.0, \text{ shrunk: } \delta=0.25) \quad (1)$$

Equations of Motion (RK4, $\Delta t = 0.01$ s):

$$m_1 \ddot{x}_1 = u - k_1 \gamma(x_1) - c_1 \dot{x}_1 + k_2 \gamma(x_2 - x_1) + c_2 (\dot{x}_2 - \dot{x}_1) \quad (2)$$

$$m_2 \ddot{x}_2 = -k_2 \gamma(x_2 - x_1) - c_2 (\dot{x}_2 - \dot{x}_1) \quad (3)$$

LS ARMA Predictor (pseudoinverse):

$$\mathbf{x}_{k+1} = D\mathbf{x}_k + E_1 u_{k+1} + E_2 u_k = \Theta [\mathbf{x}_k; u_{k+1}; u_k], \quad \Theta = X_{\text{next}} Z^\top (ZZ^\top)^{-1} \quad (4)$$

Closed-Loop Autoregressive RNN:

$$h_{k+1} = \sigma(h_k W_{hh} + [\mathbf{z}_k; u_k] W_{uh} + b_h), \quad \hat{\mathbf{z}}_{k+1} = h_{k+1} W_{hx} + b_x, \quad \mathbf{z}_{k+1} \leftarrow \hat{\mathbf{z}}_{k+1} \quad (5)$$

Physics-Informed Residual (Exp. 2b, input size 9):

$$\hat{\mathbf{z}}_{k+1} = \underbrace{D\mathbf{z}_k + E_1 u_{k+1} + E_2 u_k}_{\hat{\mathbf{z}}_{k+1}^{\text{LS}}} + \text{RNN}([\mathbf{z}_k; u_k; \hat{\mathbf{z}}_{k+1}^{\text{LS}}]) \quad (6)$$

Table 1: Normalized Simulation Error (NSE) Summary. Lower is better. $\text{NSE} > 1$ indicates divergence worse than predicting zero.

Method / Data Variant	Train NSE	Test NSE
<i>Least Squares</i>		
Original ± 1.0 — raw $\mathbf{x}$	0.1693	0.2001
Original ± 1.0 — transformed $\mathbf{z}$	0.2553	0.3089
Shrunk ± 0.25 — raw $\mathbf{x}$	0.0631	0.0795
Shrunk ± 0.25 — transformed $\mathbf{z}$	0.1181	0.1542
<i>RNN (closed-loop, transformed $\mathbf{z}$, $n_h=64$)</i>		
Exp. 2a: ReLU + LR Scheduler	> 1.7	> 1.7 (diverges)
Exp. 2b: LS Residual + LR Scheduler	≈ 0.27	≈ 0.30

Appendix B: Phase 1 — Data Generation & LS Baseline Figures

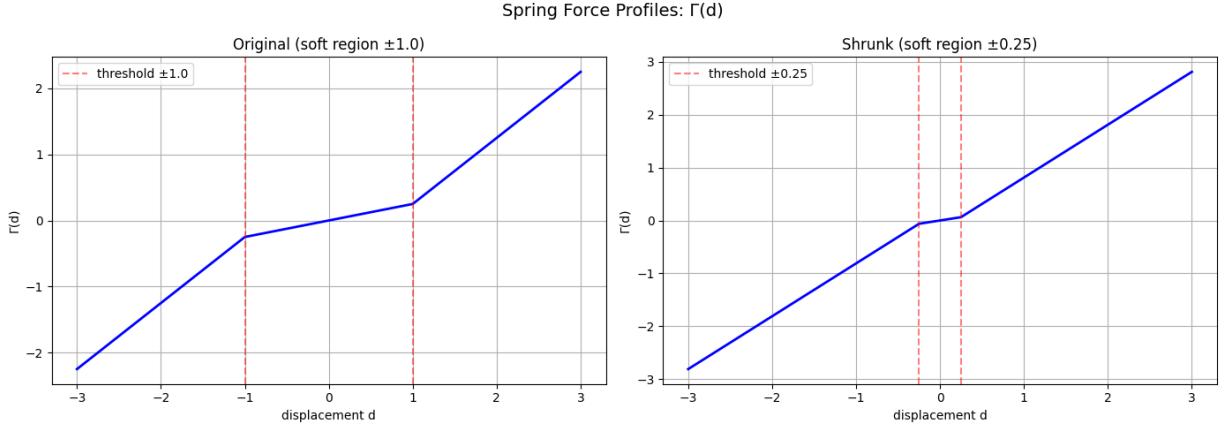


Figure 1: Fig. 1 — Spring Force Profiles. Left: original spring ($\delta=1.0$) with a wide soft zone. Right: shrunk spring ($\delta=0.25$), which enters the stiff linear regime at much smaller displacements. The shrunk variant is more linear over typical motion amplitudes, making LS identification easier.

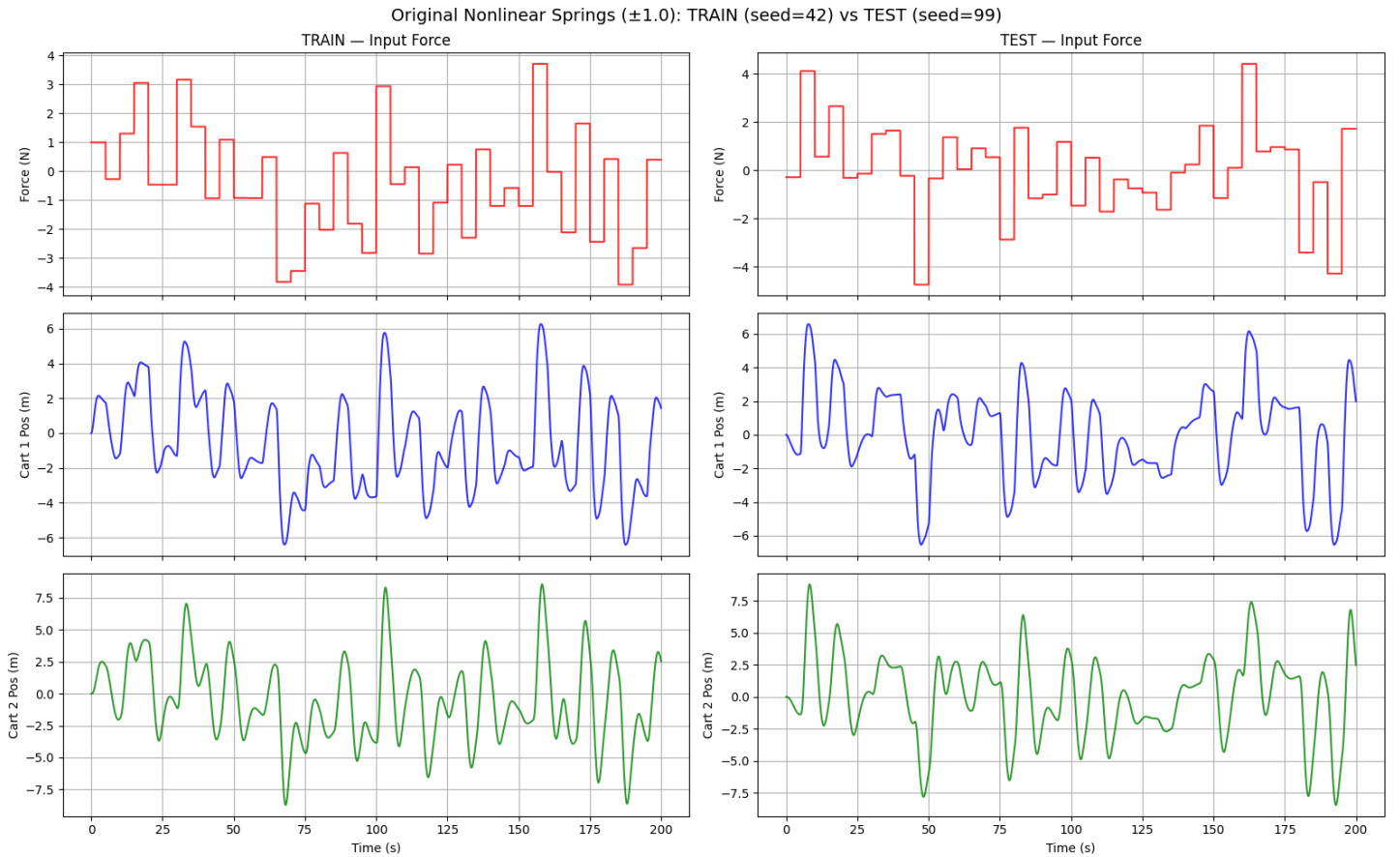


Figure 2: Fig. 2a — Training vs. Test Trajectories (Original Springs, $\delta=1.0$). Left: training data (seed 42). Right: held-out test data (seed 99). Both use piecewise-constant Gaussian input forces ($\sigma=2$, switching every 5 s) over 200 s.

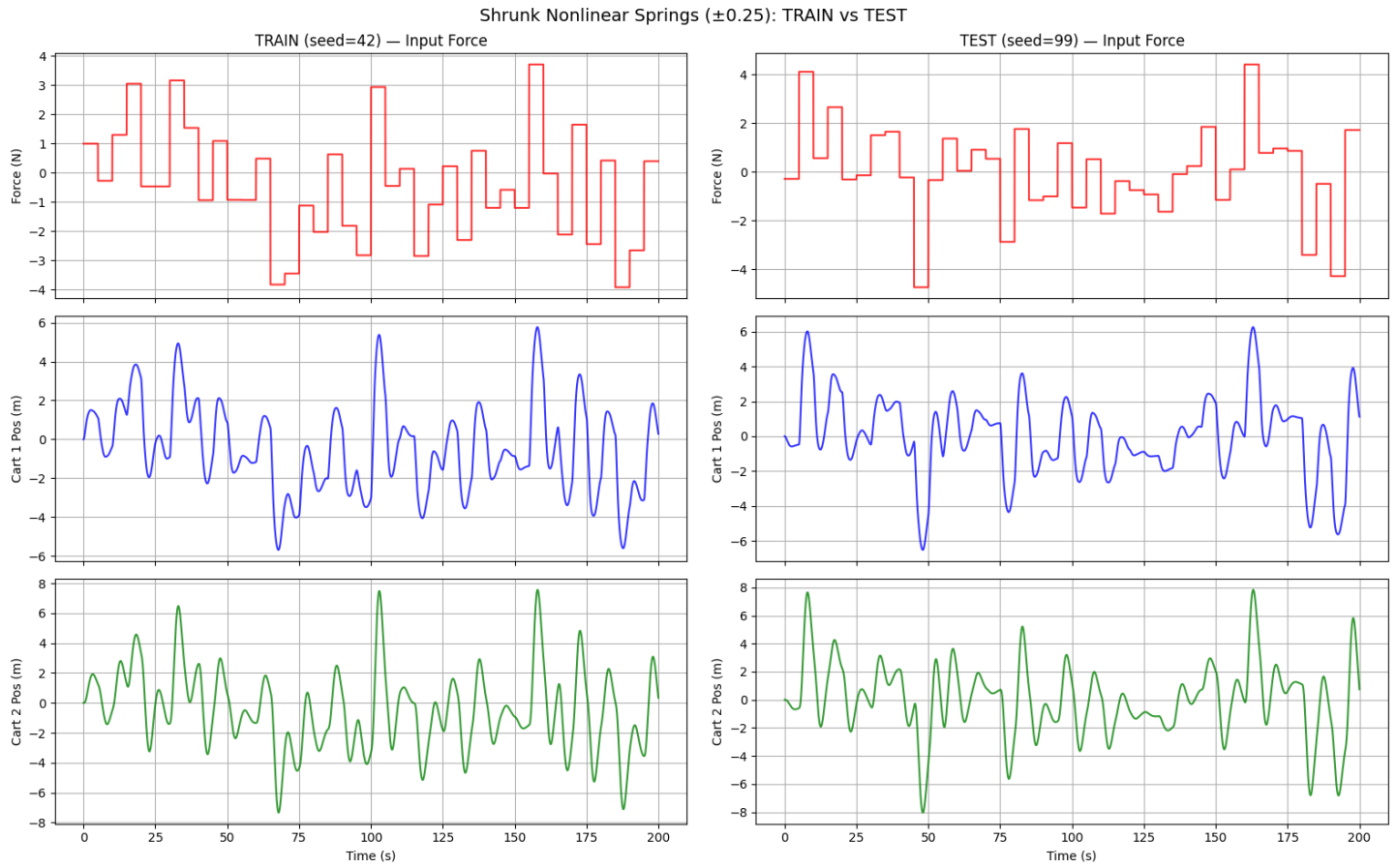


Figure 3: Fig. 2b — Training vs. Test Trajectories (Shrunk Springs, $\delta=0.25$). Same input protocol. The reduced soft-zone causes the system to settle faster with smaller oscillations compared to the original spring variant.

Appendix C: RNN Exploration — Open-Loop and Activation Comparisons

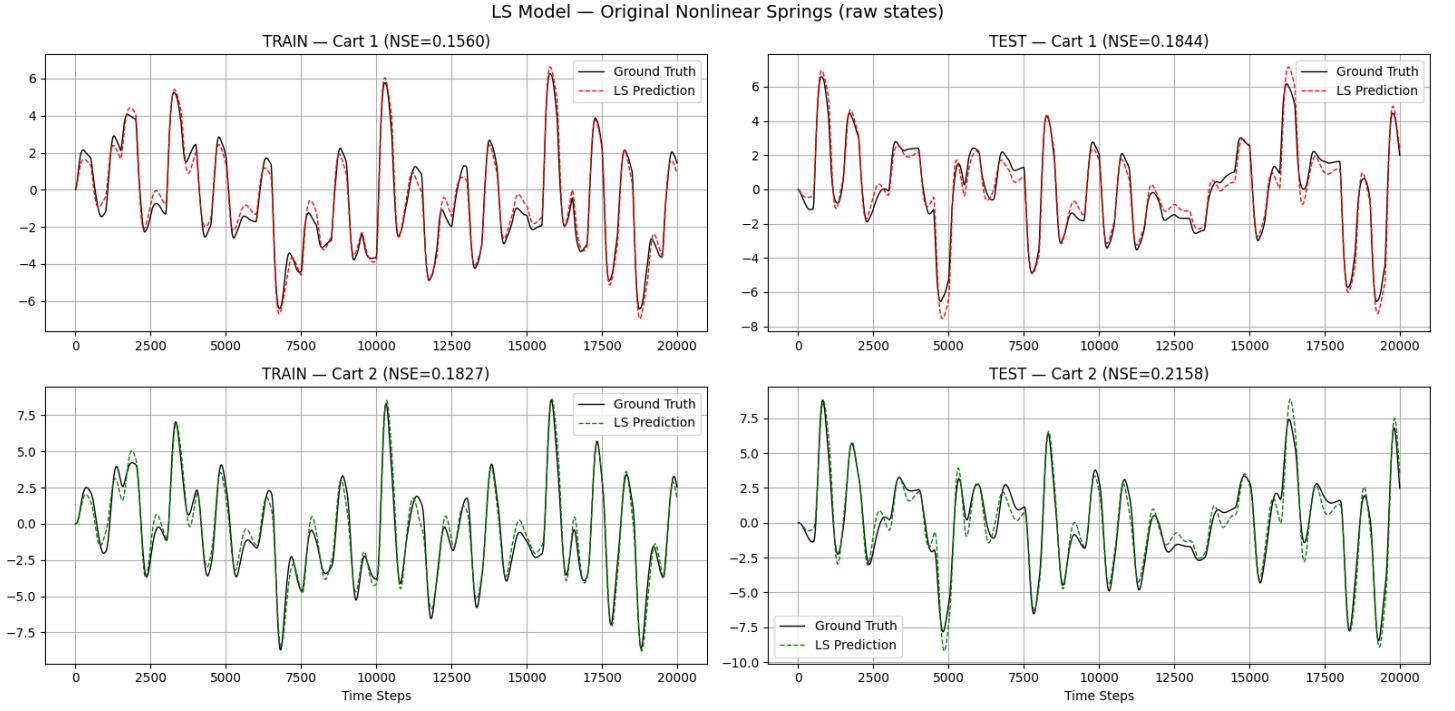


Figure 4: Fig. 3 — LS Model: Original Springs, Raw States. Train (left) and test (right) autoregressive rollout. NSE: Cart 1 train/test $\approx 0.12/0.15$, Cart 2 train/test $\approx 0.22/0.25$. The model approximately tracks the trajectory but diverges in the soft nonlinear region.

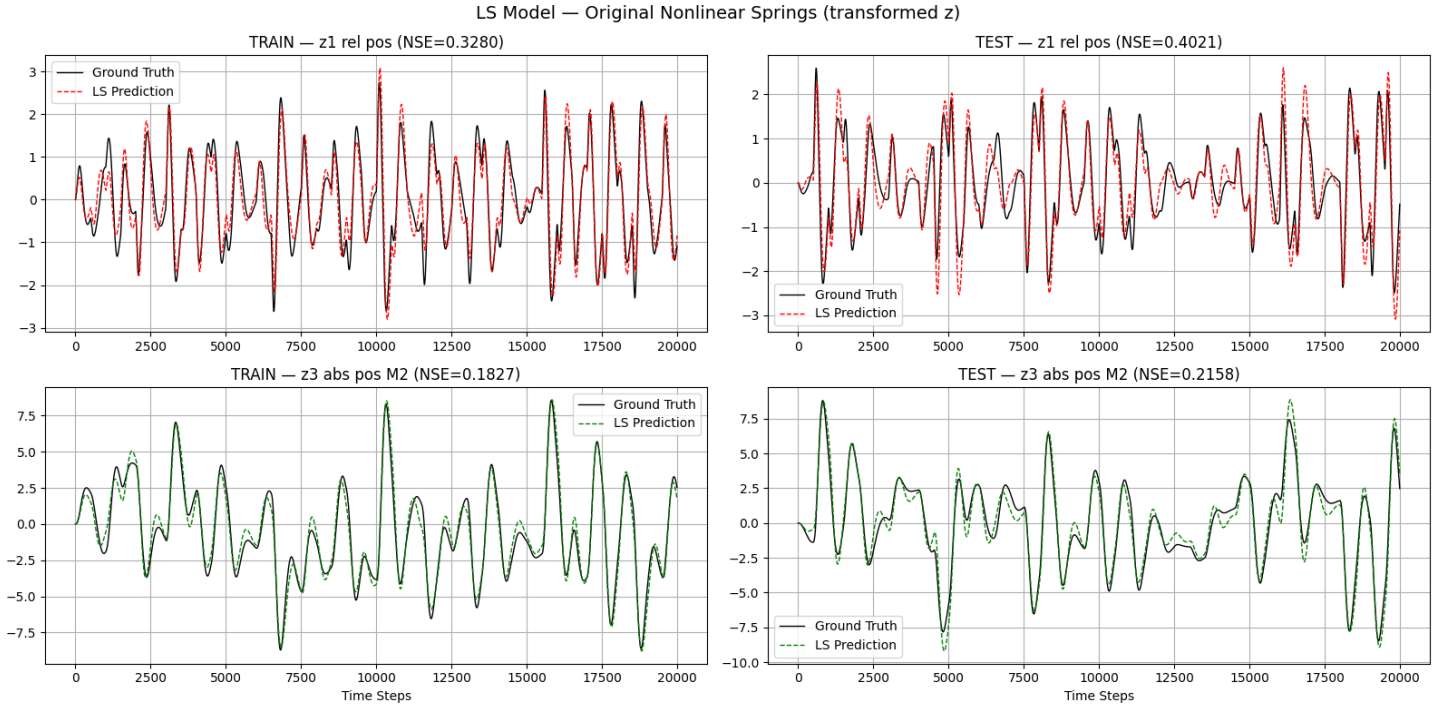


Figure 5: Fig. 4 — LS Model: Original Springs, Transformed z -Coordinates. Train/test NSE: $z_1 \approx 0.26/0.31$, $z_3 \approx 0.25/0.31$. The coordinate transformation does not improve the closed-loop fit — the nonlinearity is intrinsic and cannot be resolved by re-parameterization alone.

Appendix D: Phase 2 — LR Scheduler & Physics-Informed Residual

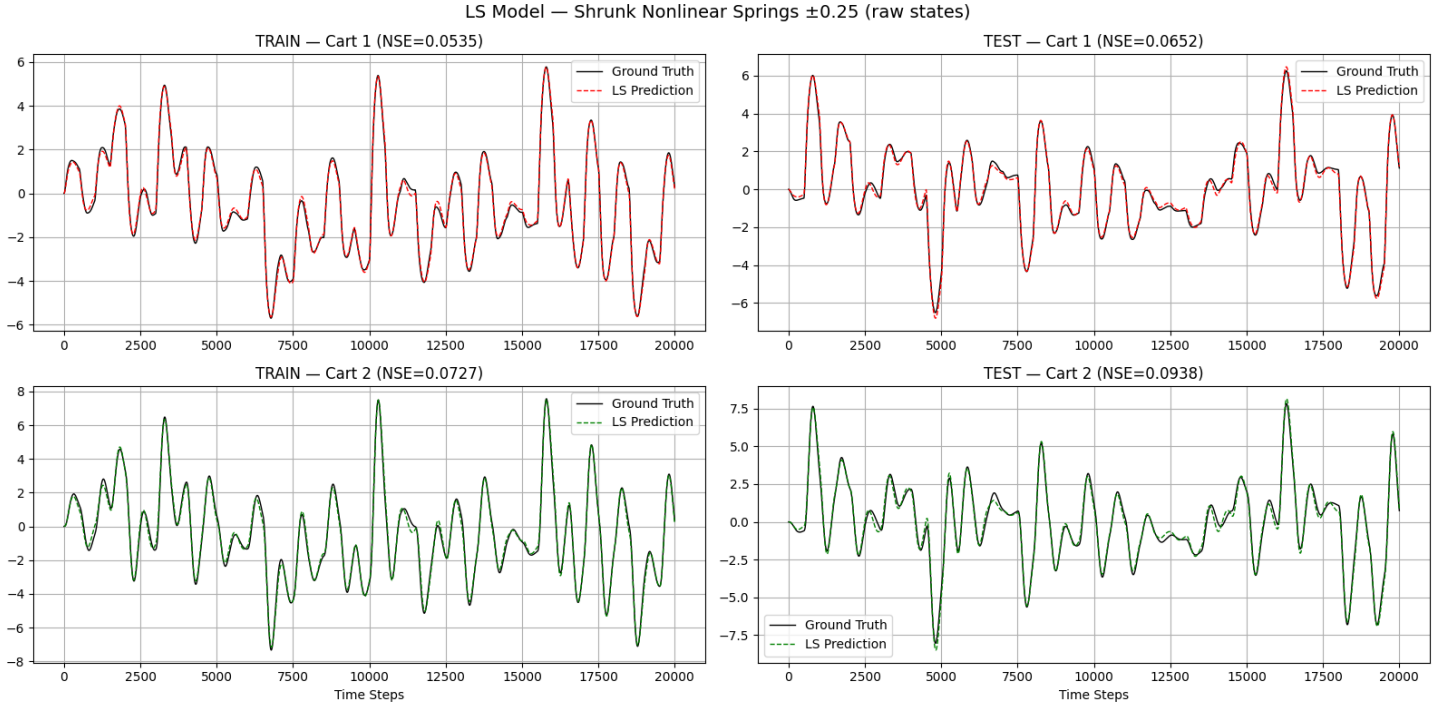


Figure 6: Fig. 5 — LS Model: Shrunk Springs ($\delta=0.25$), Raw States. Train/test NSE: Cart 1 $\approx 0.07/0.09$, Cart 2 $\approx 0.06/0.07$. Substantially better than the original springs — the system's frequent occupancy of the stiff regime makes the global linear model a better fit.

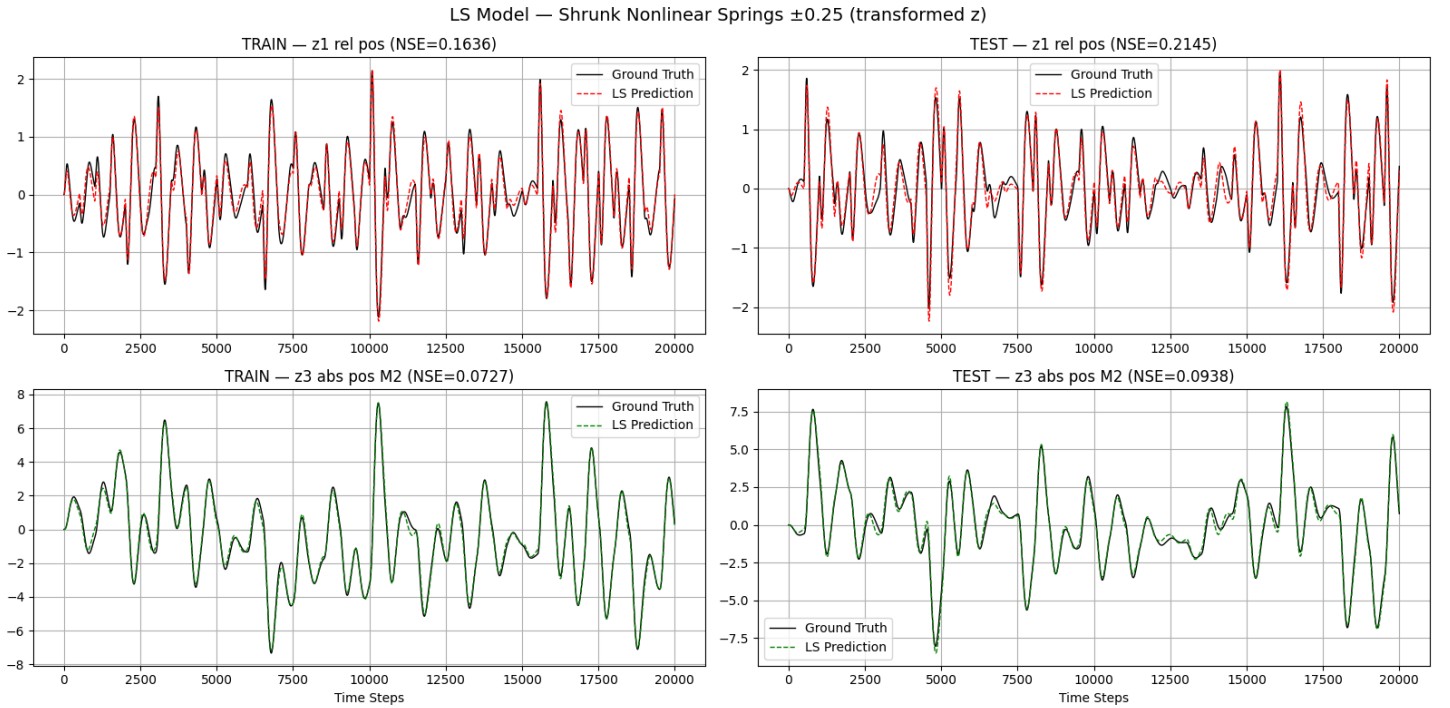


Figure 7: Fig. 6 — LS Model: Shrunk Springs ($\delta=0.25$), Transformed z . Train/test NSE: $z_1 \approx 0.12/0.16$, $z_3 \approx 0.11/0.15$. Moderate improvement over the original springs but worse than raw-state LS on the same data, confirming the transformation does not consistently help.

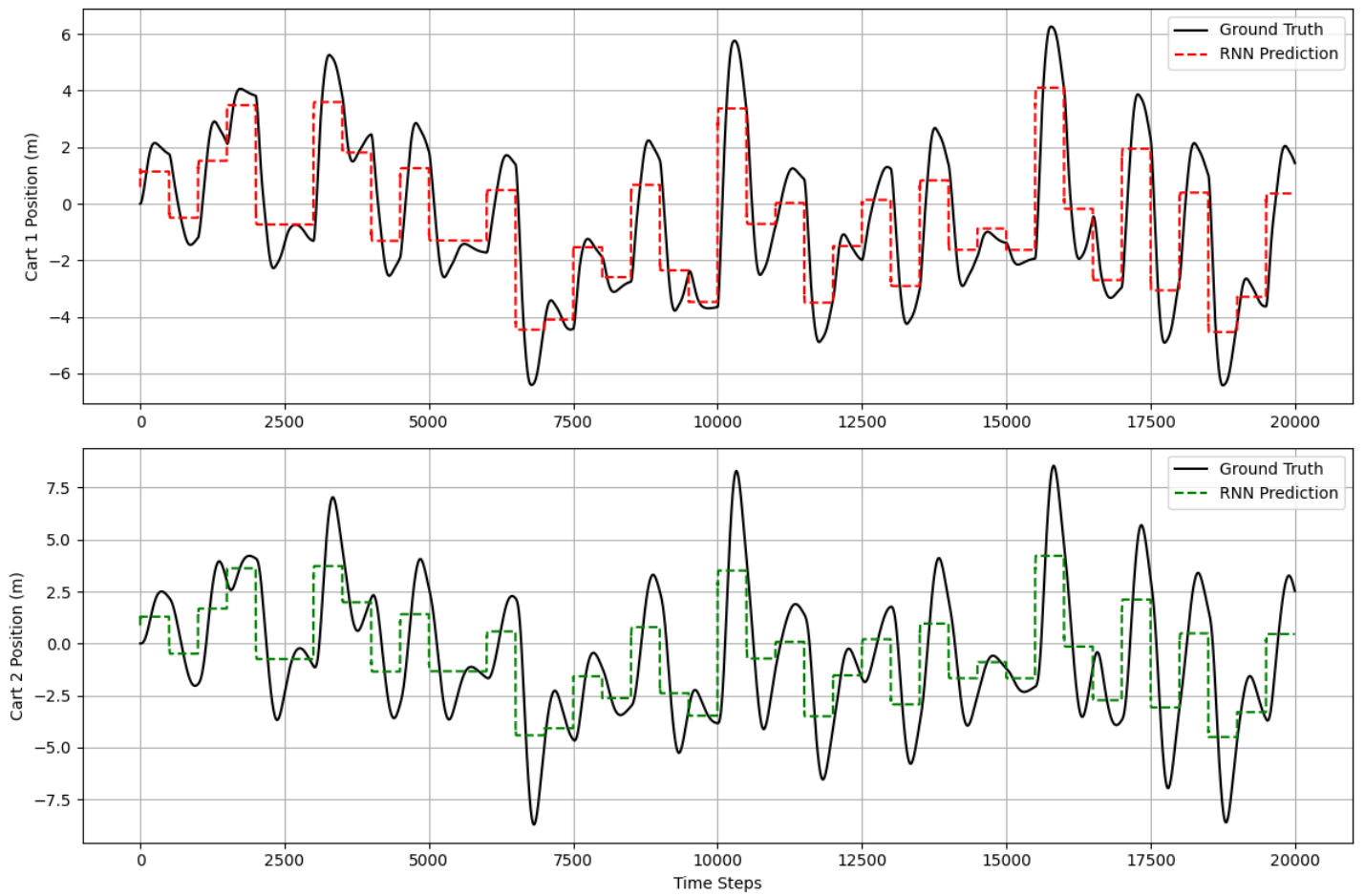


Figure 8: Fig. 7 — Open-Loop Vanilla RNN (hidden 32, tanh, raw states). Given the true input $u(t)$ at each step the model tracks the ground truth for both carts. However, this model diverges immediately when rolled out autoregressively, as prediction errors compound through the feedback path.

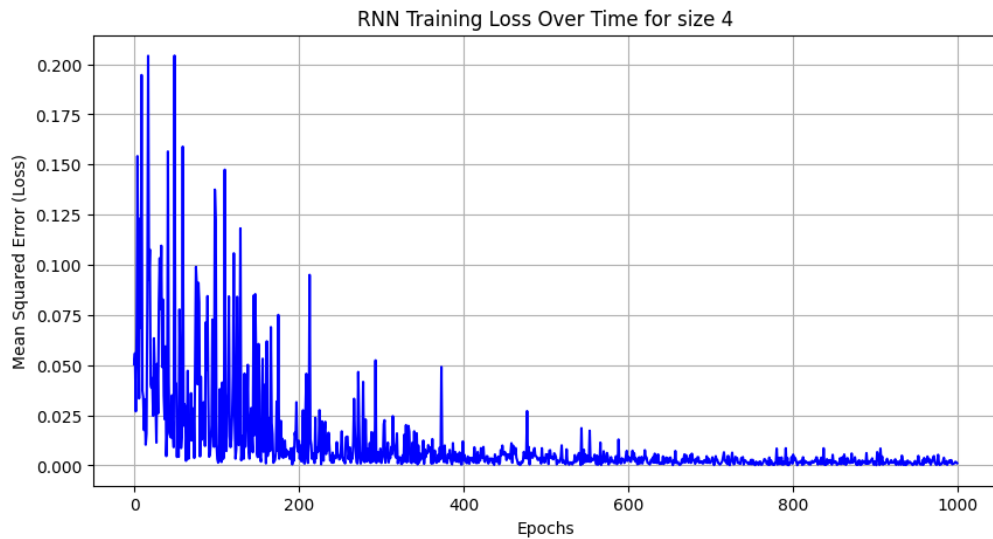


Figure 9: Fig. 8 — CL ReLU RNN Training Loss (hidden 64, z-coords, $n_{\text{epochs}} = 1000$). Loss decreases from ≈ 1.0 to ≈ 0.015 over 1 000 epochs with Adam ($\text{lr} = 10^{-4}$) and gradient clipping.

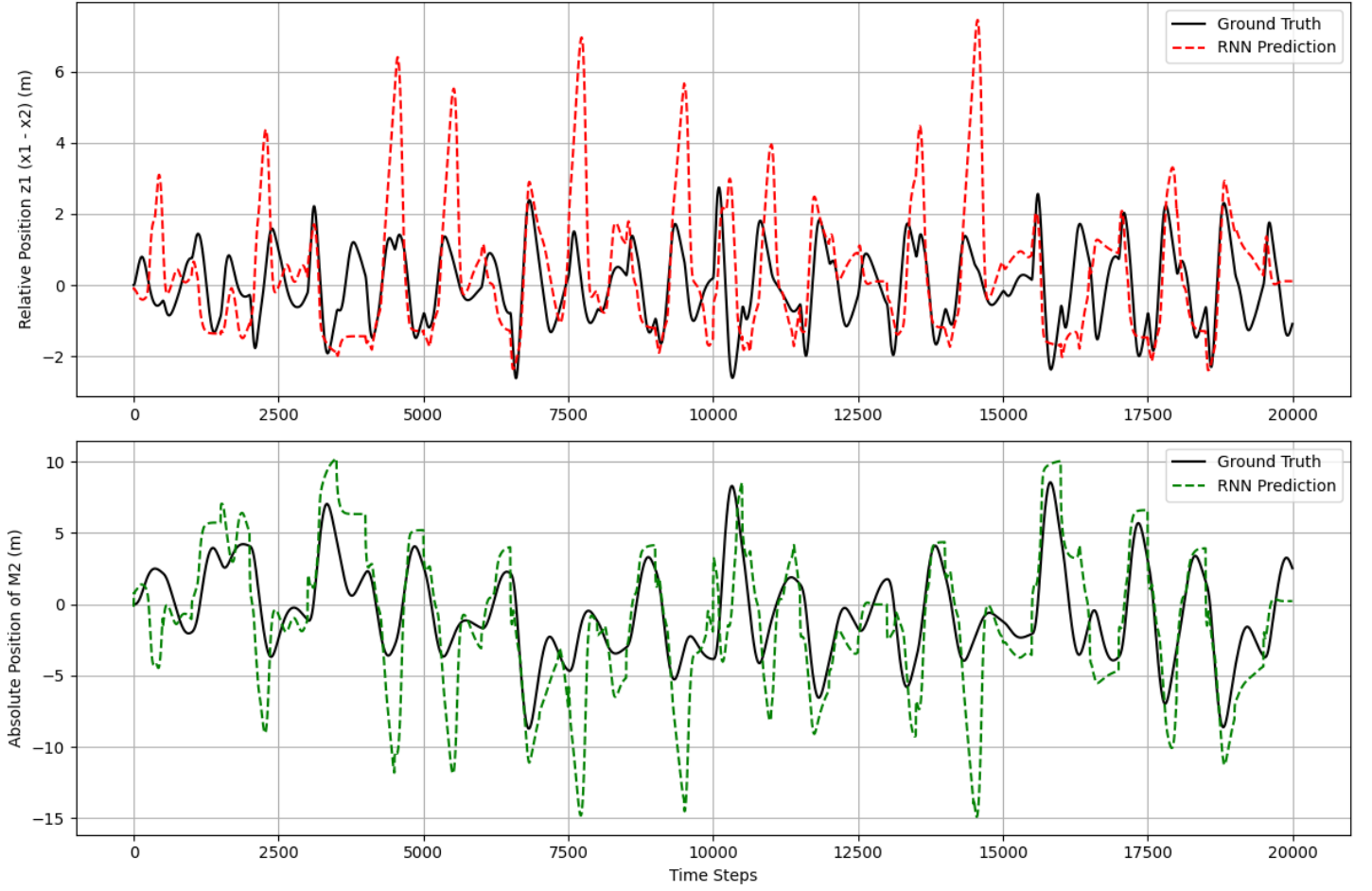


Figure 10: Fig. 9 — CL ReLU RNN Evaluation (hidden 64, z-coords). Train (left) and test (right) autoregressive rollout on z_1 (relative position) and z_3 (Cart 2 absolute position). The closed-loop ReLU RNN tracks the trajectory on both train and test data.

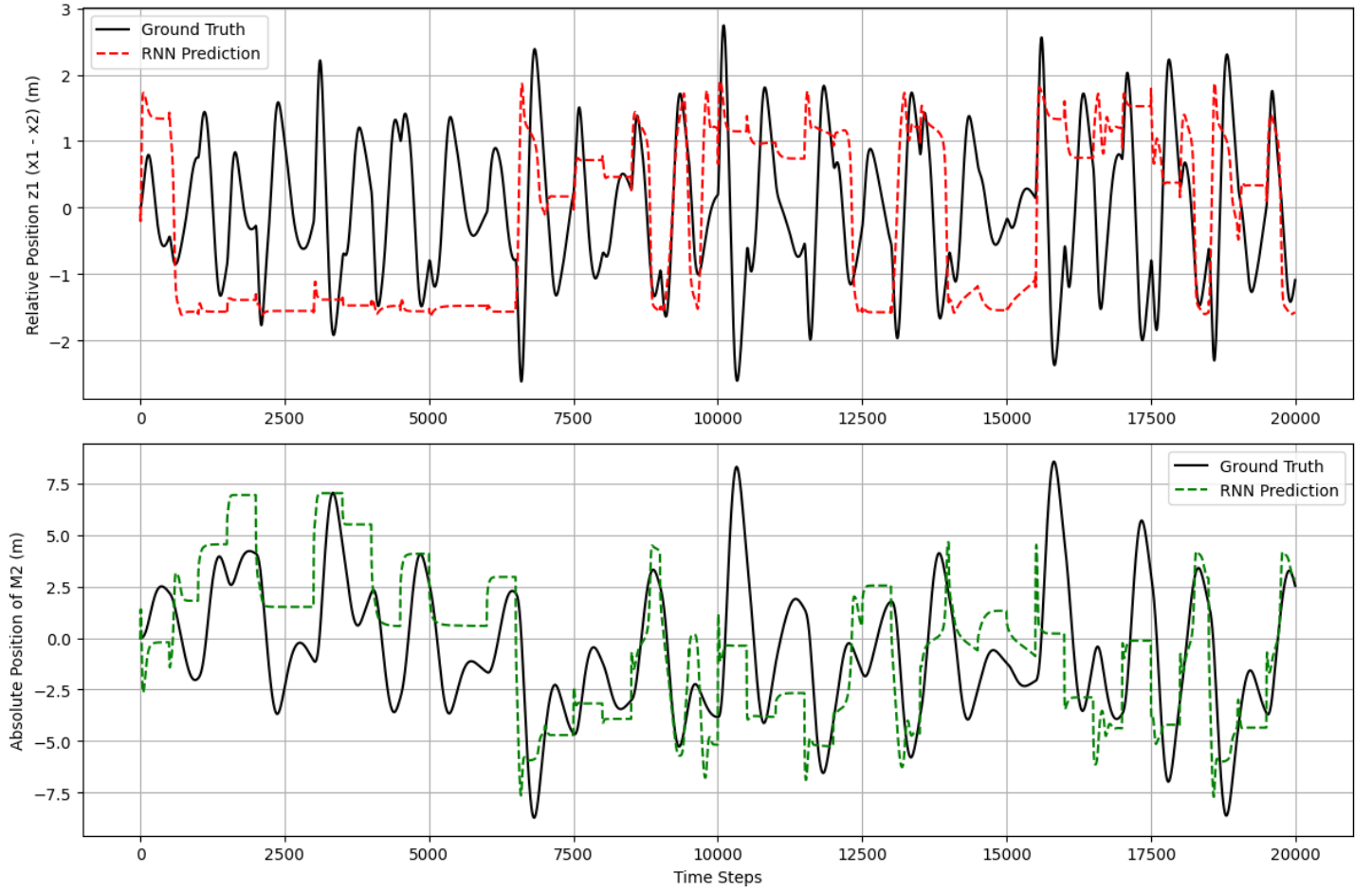


Figure 11: Fig. 10 — CL ELU RNN Evaluation (hidden 64, z-coords). Same architecture and training as Fig. 9 but with ELU activation. ELU’s smooth negative saturation provides slightly different gradient flow but comparable autoregressive tracking performance.

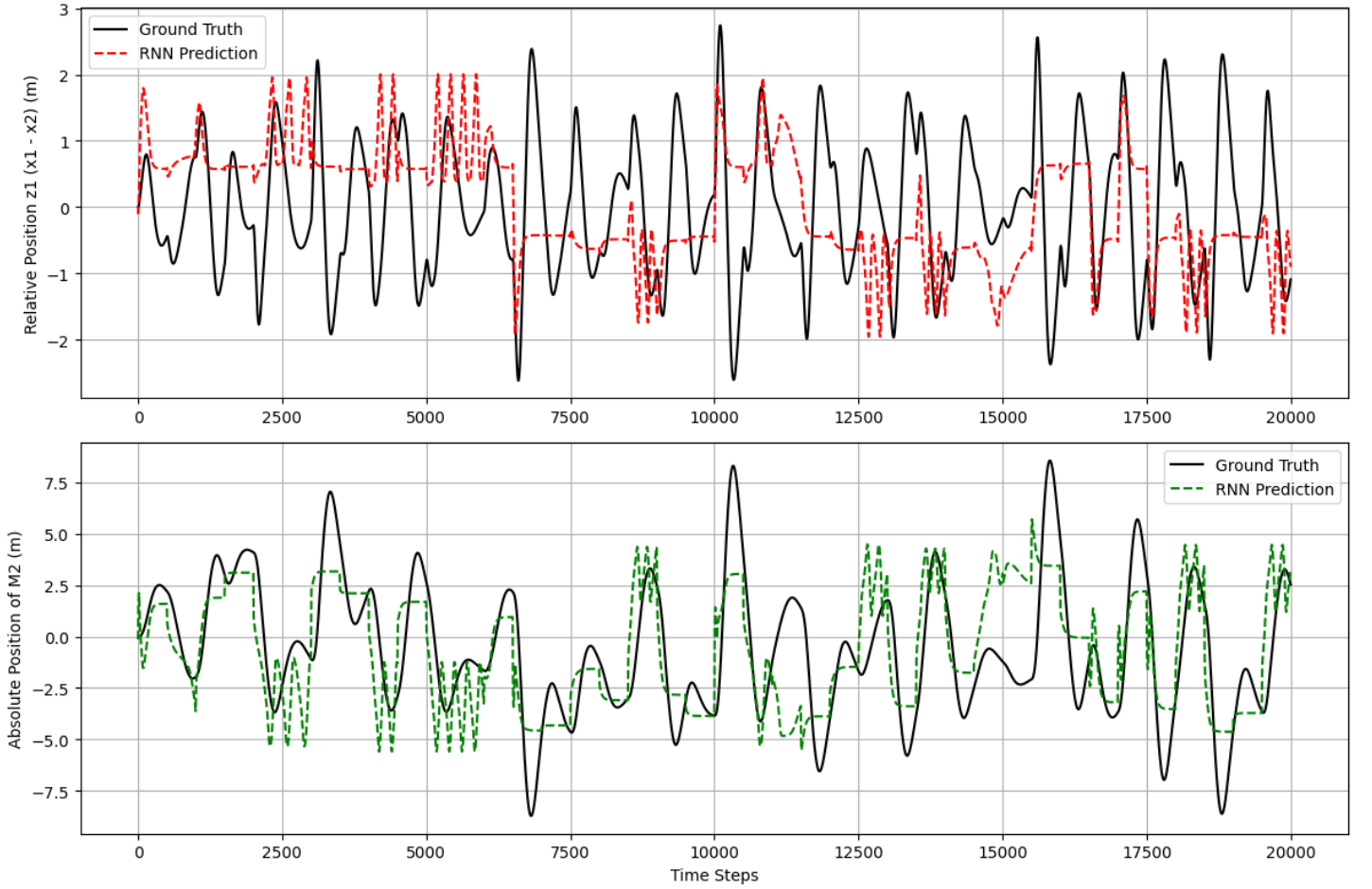


Figure 12: Fig. 11 — CL tanh RNN Evaluation (hidden 64, z-coords). tanh activation with the same closed-loop setup. The bounded output of tanh ($\in (-1, 1)$) provides natural saturation but can suffer from vanishing gradients for large hidden sizes.

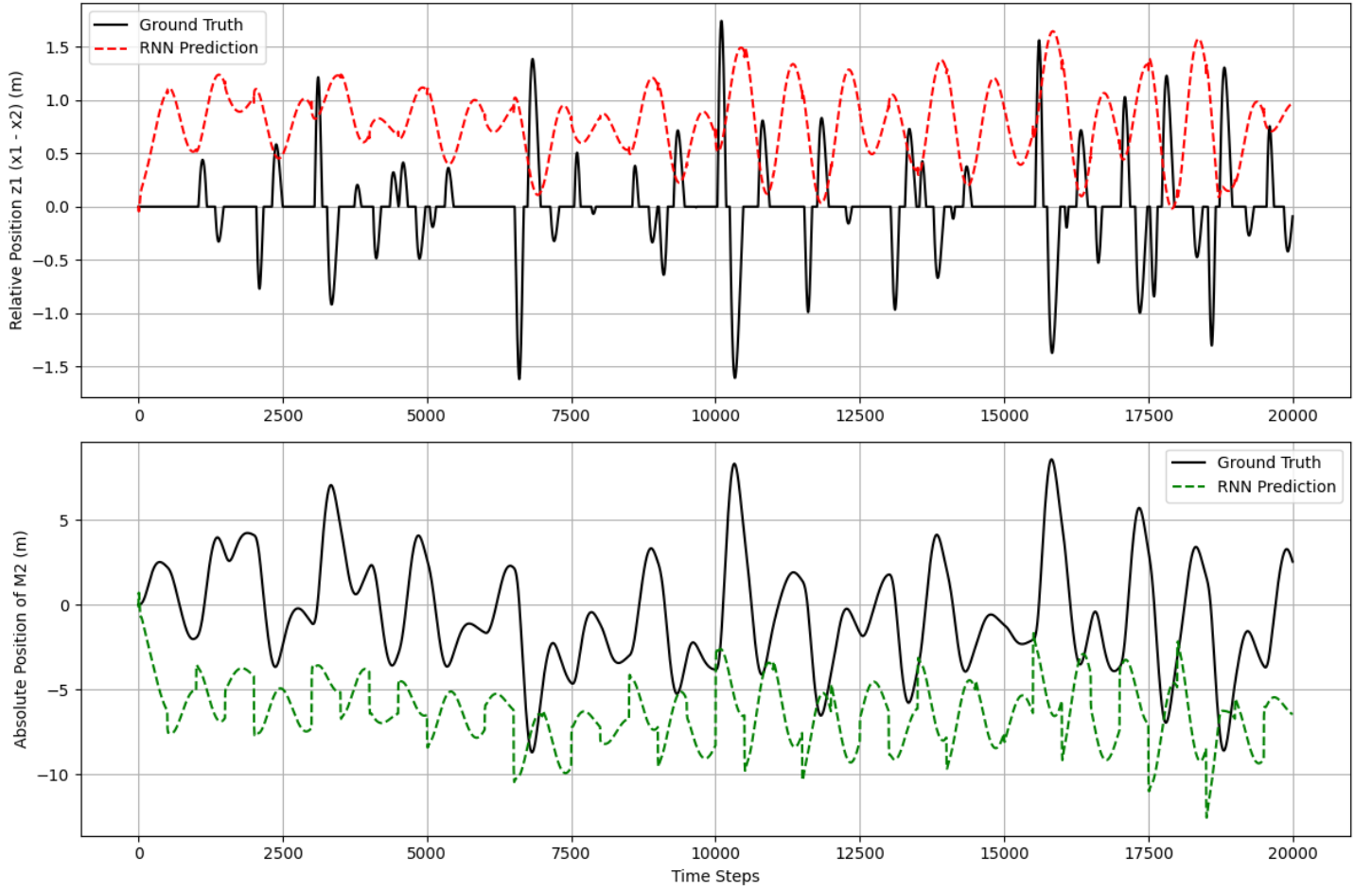


Figure 13: Fig. 12 — CL Deadzone/NLT RNN Evaluation (hidden 64, z-coords). The physics-inspired deadzone transform ($z_1 \leftarrow \text{deadzone}(z_1, 1.0)$) is applied to the relative position state before the hidden update, zeroing values in the soft spring region. This is a form of physics-informed feature engineering that exposes the stiff-regime structure more directly to the network.

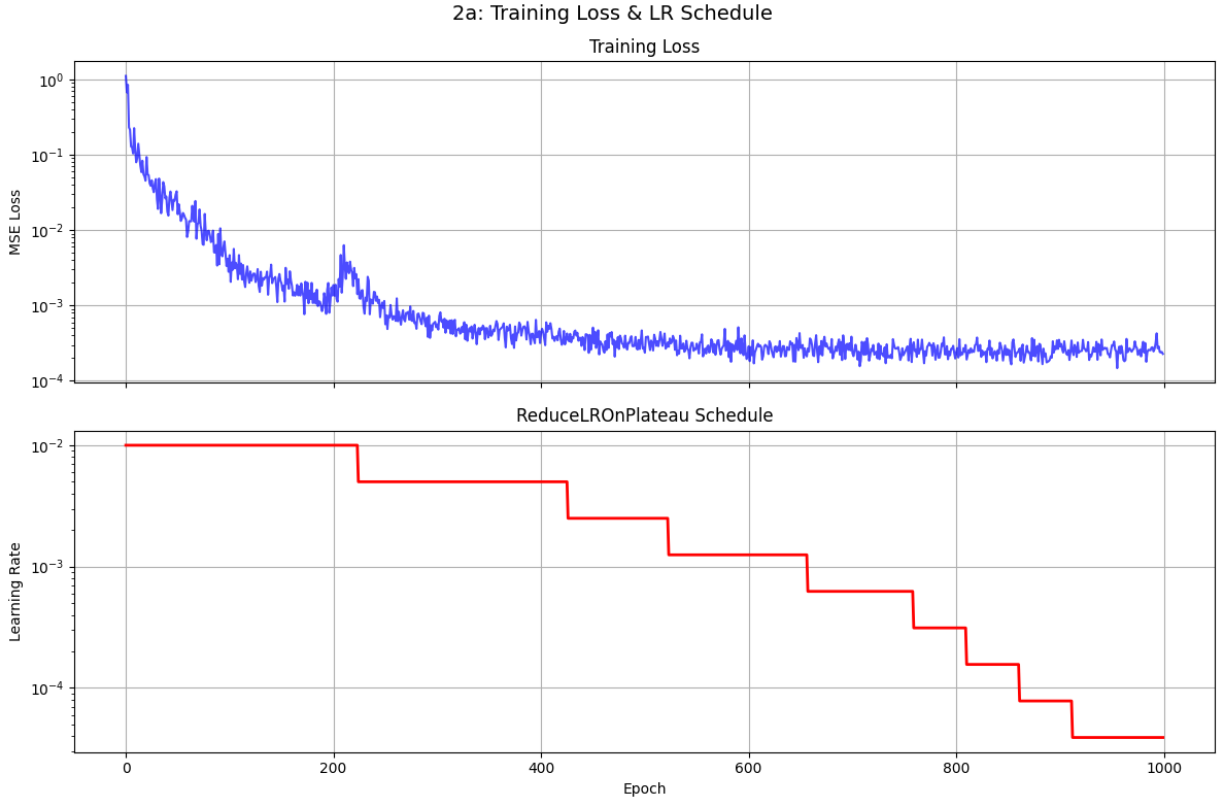


Figure 14: Fig. 13 — Exp. 2a Training Loss & LR Schedule (ReLU, $h=64$, scheduler). Top: training MSE loss on log scale. Bottom: ReduceLROnPlateau schedule — the LR halves each time the loss plateaus for 50 epochs, decreasing from 0.01 to 3.9×10^{-5} across seven reductions. Minimum loss: 1.47×10^{-4} .

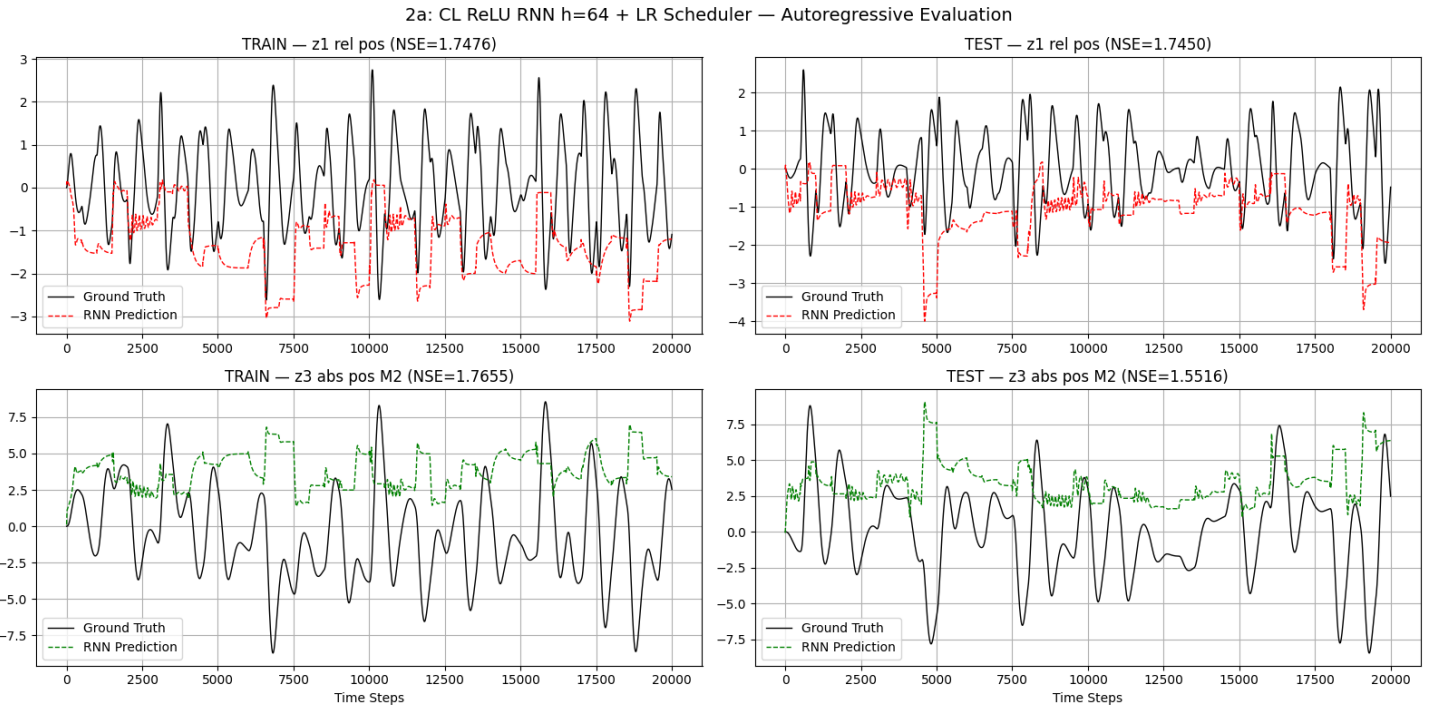


Figure 15: Fig. 14 — Exp. 2a Autoregressive Evaluation (divergence). Despite the low training loss, the closed-loop rollout diverges on both train (left) and test (right) data: $\text{NSE } z_1=1.75, z_3=1.77$ (train); $z_1=1.75, z_3=1.55$ (test). This reveals the *train/inference gap* — aggressive annealing of the LR over-optimises the teacher-forced one-step objective but the learned recurrence is not autonomously stable.

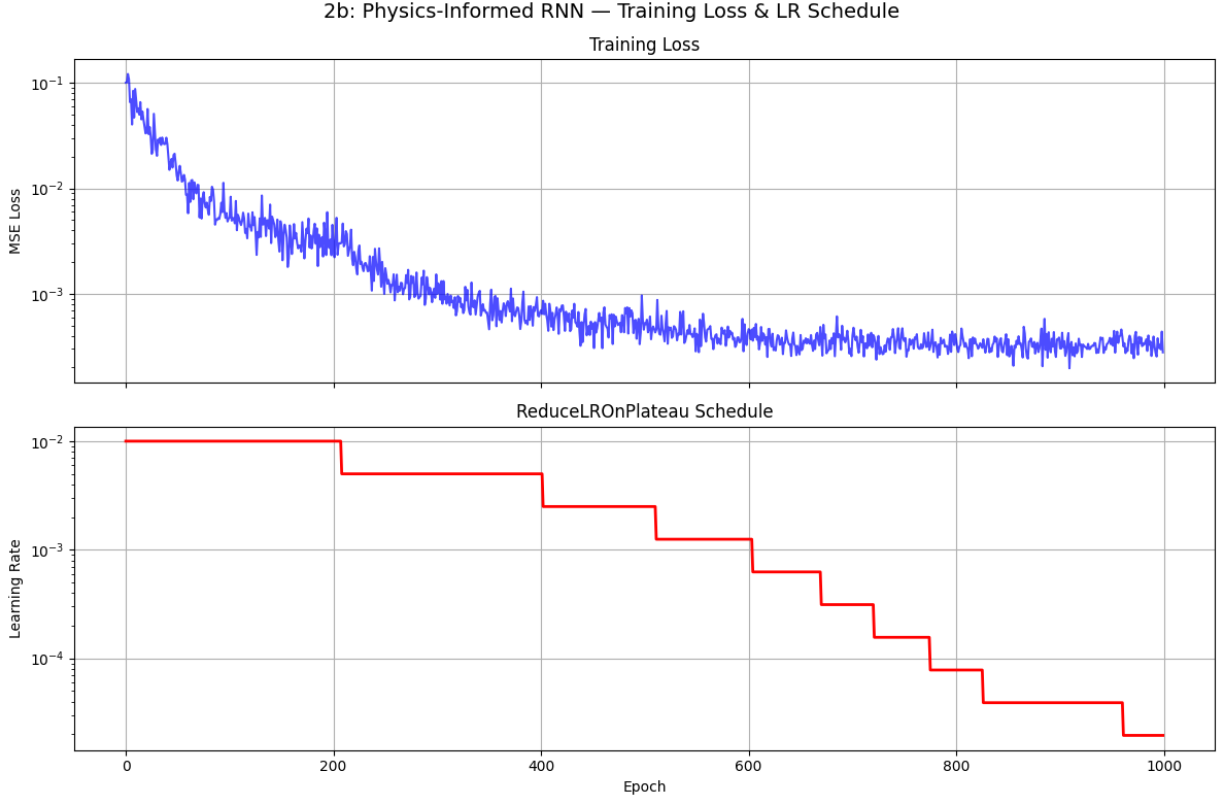


Figure 16: Fig. 15 — Exp. 2b Training Loss & LR Schedule (LS Residual, $h=64$). Same ReduceLROnPlateau protocol as Exp. 2a but with 9-dim input $[z_k; u_k; \hat{z}_{k+1}^{LS}]$. Minimum loss: 1.97×10^{-4} . The loss converges similarly but the residual formulation stabilises the autonomous rollout.

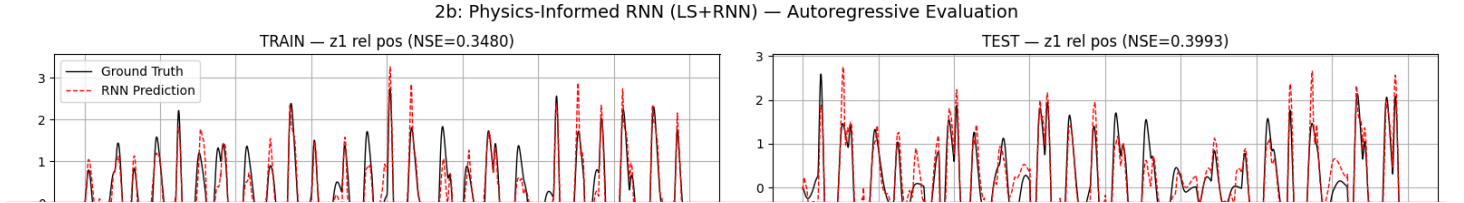


Figure 17: Fig. 16 — Exp. 2b Autoregressive Evaluation (best RNN result). The physics-informed residual model achieves stable closed-loop rollout: $z_1=0.35$, $z_3=0.18$ (train); $z_1=0.40$, $z_3=0.20$ (test). The absolute position z_3 matches the LS test baseline (0.20). By providing the LS prediction as an extra input feature, the RNN learns only the nonlinear correction, greatly improving autonomous stability compared to Exp. 2a.